

De la teoría a la práctica: walkthrough en acción (Guia)

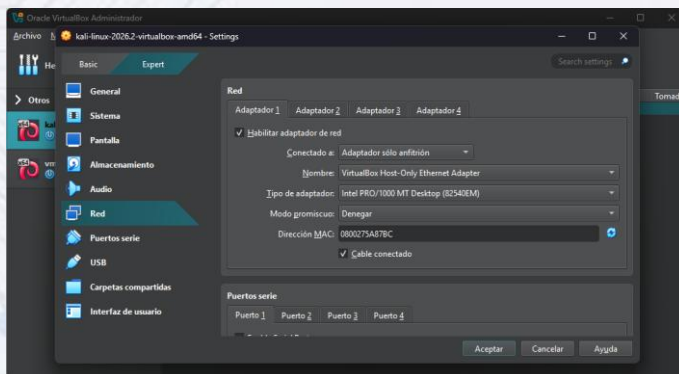
Objetivo general:

Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado y la explicación de las técnicas y metodologías utilizadas para vulnerar una máquina virtual.

Configurar un entorno de virtualización

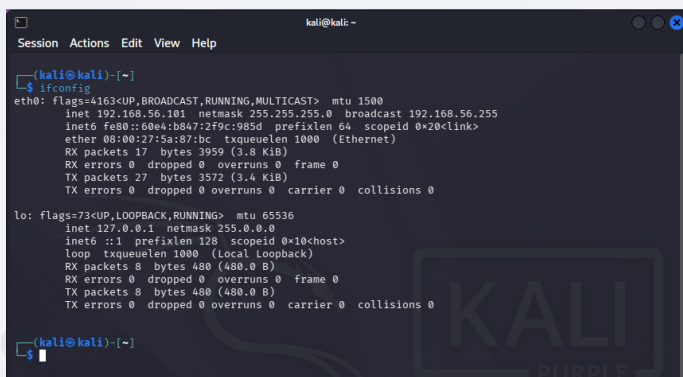
Para configurar un entorno de virtualización aislado tenemos que configurar los adaptadores red de la maquina vulnerable (Snake oil) y de la maquina en la cual vamos a hacer las pruebas de Pentesting que en este caso es una maquina con Kali Linux

En VirtualBox realizaremos el siguiente proceso en cada una de las máquinas virtuales, clic derecho a la maquina a configurar, dar clic al apartado red, del lado derecho encontraremos la opción conectada a, aquí seleccionamos la opción: adaptador solo anfitrión, esta opción nos genera una red interna en la cual solo la maquina Snake oil y Kali Linux estarían conectadas sin acceso a internet. Una vez realizado este proceso encendemos las maquinas.



Enumerar los servicios y aplicaciones en ejecución

En Kali Linux empezamos identificando la dirección Ip de nuestra maquina abriendo la terminal y ejecutando el comando ifconfig.

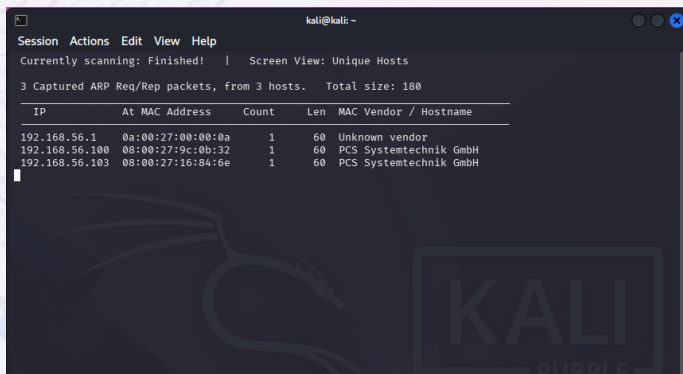


```
kali@kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.56.101  netmask 255.255.255.0  broadcast 192.168.56.255
    inet6 fe80::d0e4:b847:2f9c:985d  prefixlen 64  scopeid 0x20<link>
    ether 08:00:27:5a:87:bc  txqueuelen 1000  (Ethernet)
    RX packets 17  bytes 3959 (3.8 KiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 27  bytes 3572 (3.4 KiB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (local loopback)
    RX packets 8  bytes 480 (480.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 8  bytes 480 (480.0 B)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
```

Ahora necesitamos encontrar la dirección de la maquina Snake oíl para esto usamos el comando **netdiscover** este trabaja enviando petición **broadcast** la cual responden todos los hosts menos el que la envía dentro de una red con su Mac y dirección Ip asociada, en sus opciones tendremos que ingresar la interfaz que estamos usando y el rango de direcciones que estamos buscando de la siguiente forma.

Sudo netdiscover -i eth0 -r 192.168.56.0/24



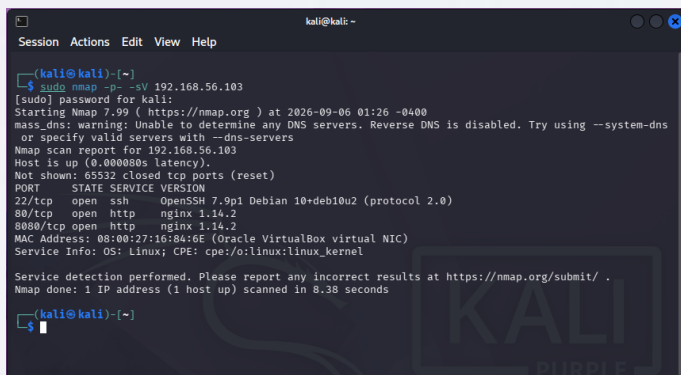
```
kali@kali:~$ sudo netdiscover -i eth0 -r 192.168.56.0/24
Currently scanning: Finished! | Screen View: Unique Hosts
3 Captured ARP Req/Rep packets, from 3 hosts. Total size: 180

  IP             At MAC Address  Count  Len  MAC Vendor / Hostname
  -----
192.168.56.1     0a:00:27:00:00:0a  1      60   Unknown vendor
192.168.56.100   08:00:27:9c:0b:32  1      60   PCS Systemtechnik GmbH
192.168.56.103   08:00:27:16:84:6e  1      60   PCS Systemtechnik GmbH
```

Aquí podemos ver tres direcciones Ip descartando la primera que pertenece al adaptador de red interno de VirtualBox y la segunda al servidor DHCP de VirtualBox. Descubrimos que la dirección: **192.168.56.103** pertenece a la maquina Snake oíl.

Una vez que tenemos la dirección tendremos que enumerar los servicios activos sobre esta Ip para eso usamos el comando **nmap** que nos ayudara escaneando todos los puertos abiertos, así como el servicio y la versión de este de la siguiente manera.

Sudo nmap -p- -sV 192.168.56.103

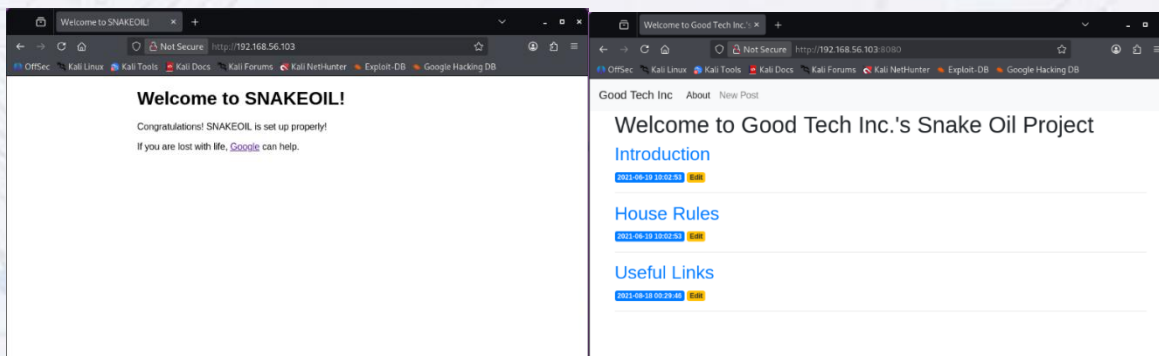


```
kali@kali: ~  
$ sudo nmap -p- -sV 192.168.56.103  
[sudo] password for kali:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 01:26 -0400  
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns  
or specify valid servers with --dns-servers  
Nmap scan report for 192.168.56.103  
Host is up (0.000080s latency).  
Not shown: 65532 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)  
80/tcp    open  http     nginx 1.14.2  
8080/tcp  open  http     nginx 1.14.2  
MAC Address: 08:100:27:16:58:0E (Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 8.38 seconds  
  
kali@kali: ~  
$
```

Al analizar la información obtenida podemos ver que está abierto el servicio **shh** y los puertos **80** y **8080** los cuales me indican que se está ejecutando un **servidor web nginx** .

Vulnerabilidades conocidas asociadas con los servicios

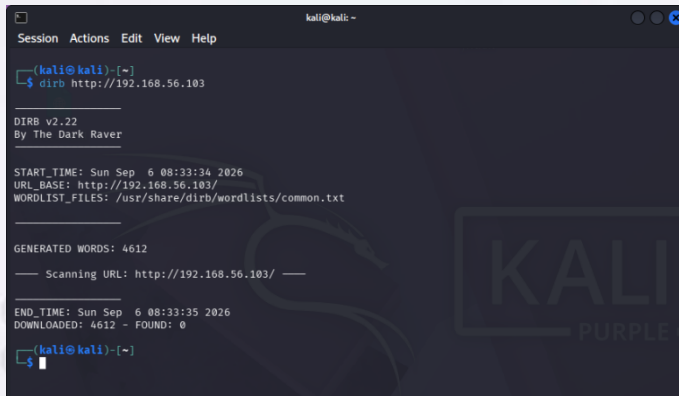
Al ingresar al servidor podemos ver páginas web simples sin protección https lo cual nos permitirá ver las peticiones y respuestas del servidor web ya que no están encriptadas.



Seleccionar y adaptar los exploits

Haremos uso de la herramienta **dirb** la cual escanea directorios ocultos dentro del servidor comparando sus nombres con una lista directorios comunes en un servidor web de la siguiente manera.

http://192.168.56.103



```
kali@kali:~$ dirb http://192.168.56.103

DIRB v2.22
By The Dark Raver

START_TIME: Sun Sep  6 08:33:34 2026
URL_BASE: http://192.168.56.103/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

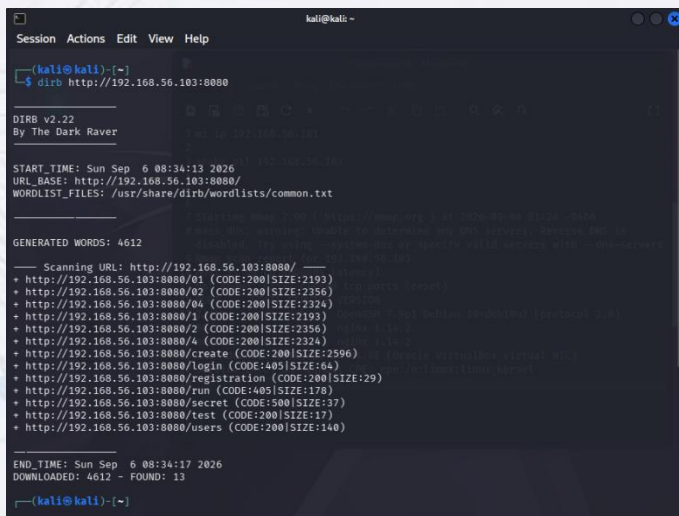
--- Scanning URL: http://192.168.56.103/ ---

END_TIME: Sun Sep  6 08:33:35 2026
DOWNLOADED: 4612 - FOUND: 0

kali@kali:~$
```

Al ejecutar el escaneo en el puerto **80** no arroja ningún directorio por lo cual este puerto queda descartado ejecutamos un escaneo por el puerto **8080** de la siguiente manera.

http://192.168.56.103:8080



```
kali@kali:~$ dirb http://192.168.56.103:8080

DIRB v2.22
By The Dark Raver

START_TIME: Sun Sep  6 08:34:13 2026
URL_BASE: http://192.168.56.103:8080/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

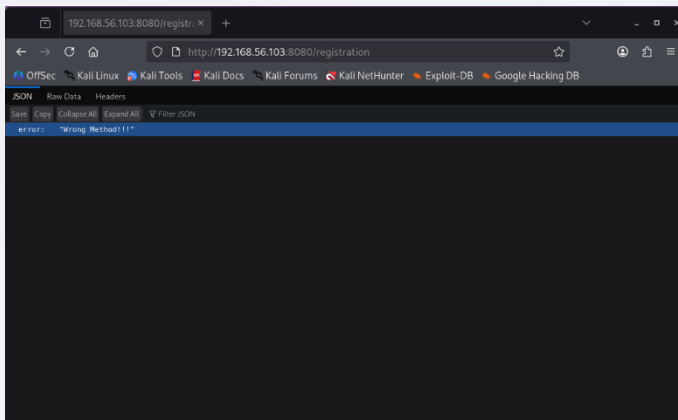
--- Scanning URL: http://192.168.56.103:8080/ ---
+ http://192.168.56.103:8080/01 (CODE:200|SIZE:2193)
+ http://192.168.56.103:8080/02 (CODE:200|SIZE:2356)
+ http://192.168.56.103:8080/04 (CODE:200|SIZE:2324)
+ http://192.168.56.103:8080/1 (CODE:200|SIZE:2193)
+ http://192.168.56.103:8080/2 (CODE:200|SIZE:2356)
+ http://192.168.56.103:8080/4 (CODE:200|SIZE:2324)
+ http://192.168.56.103:8080/create (CODE:200|SIZE:2596)
+ http://192.168.56.103:8080/login (CODE:405|SIZE:64)
+ http://192.168.56.103:8080/registration (CODE:200|SIZE:29)
+ http://192.168.56.103:8080/run (CODE:405|SIZE:176)
+ http://192.168.56.103:8080/secret (CODE:500|SIZE:37)
+ http://192.168.56.103:8080/test (CODE:200|SIZE:17)
+ http://192.168.56.103:8080/users (CODE:200|SIZE:140)

END_TIME: Sun Sep  6 08:34:17 2026
DOWNLOADED: 4612 - FOUND: 13

kali@kali:~$
```

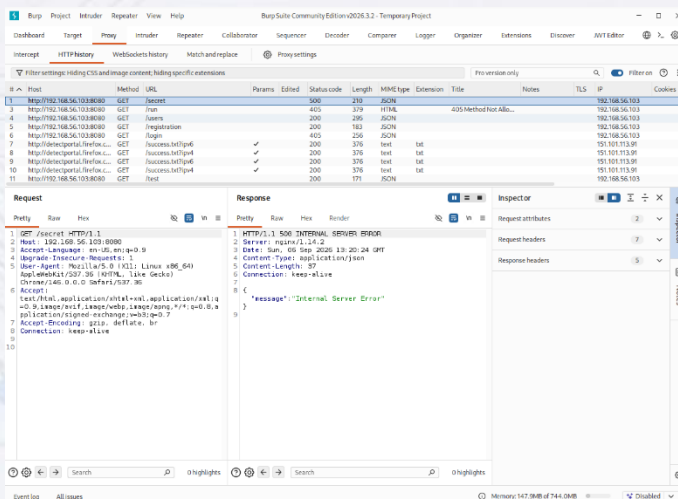
Gracias a esta Herramienta logramos encontrar directorios con nombres interesantes como **login, registration, run, secret, test, users**.

Usamos esta información para nuevamente navegar por estos directorios encontrándonos con respuestas propias de un sitio web en construcción y con funcionalidades de api por lo cual podremos interactuar con esta.



Ya que no se está usando el protocolo https, necesitaremos interceptar, analizar y modificar el tráfico entre un navegador y un servidor para detectar vulnerabilidades en esta aplicación web, por esto usaremos **Burp Suite**.

Analizaremos las peticiones y respuesta de cada sección gracias al uso del **Proxy de Burp Suite**, empezaremos por **secret** ya que por su nombre podría contener algo de suma importancia.



La respuesta a esta petición como podemos ver en la línea 4 **Content-Type: application/json** nos indica la forma en que se está interpretando los datos, por lo cual si queremos interactuar con ella tendremos que darle datos en este mismo formato aquí el mensaje nos arroja un Error de Servidor interno, este mensaje nos puede indicar que algo anda mal o que necesita algún dato para que funcione correctamente como no lo sabemos lo dejaremos por el momento.

La siguiente sección de interés es la de **users** ya que puede contener el nombre de usuario y contraseña del administrador

The screenshot shows the Burp Suite interface with the 'HTTP History' tab selected. A list of intercepted requests is shown, with the first one selected. The 'Request' pane shows the raw HTTP request, and the 'Response' pane shows the raw HTTP response. The response body is a JSON object containing a user object with the following details:

```
{
  "username": "patrick",
  "password": "2w98R4k577p9wV99L3Prm:CF66R0sP"
}
```

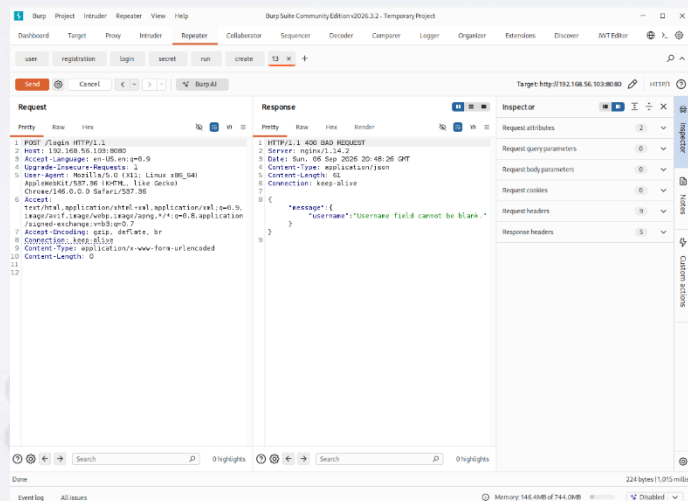
La respuesta de esta petición en el formato antes mencionado nos deja ver información relevante como un nombre de usuario “**patrick**” y una contraseña que al parecer está cifrada.

Ya que tenemos un nombre de usuario podríamos interactuar con la sección **login**.

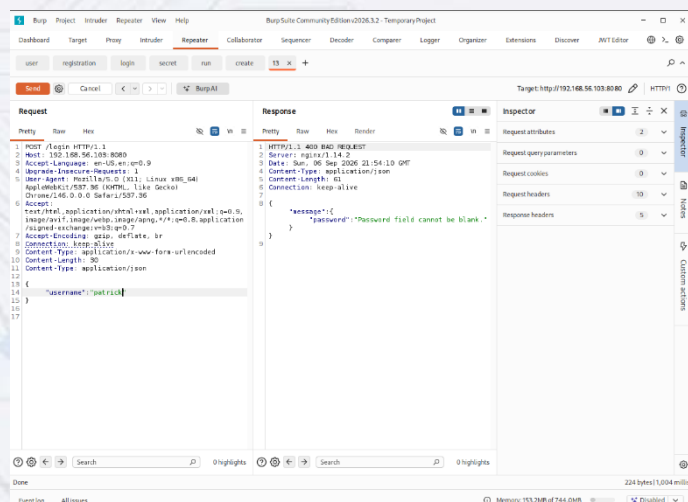
The screenshot shows the Burp Suite interface with the 'HTTP History' tab selected. A list of intercepted requests is shown, with the first one selected. The 'Request' pane shows the raw HTTP request, and the 'Response' pane shows the raw HTTP response. The response body is a JSON object containing a message:

```
{
  "message": "The method is not allowed for the requested URL."
}
```

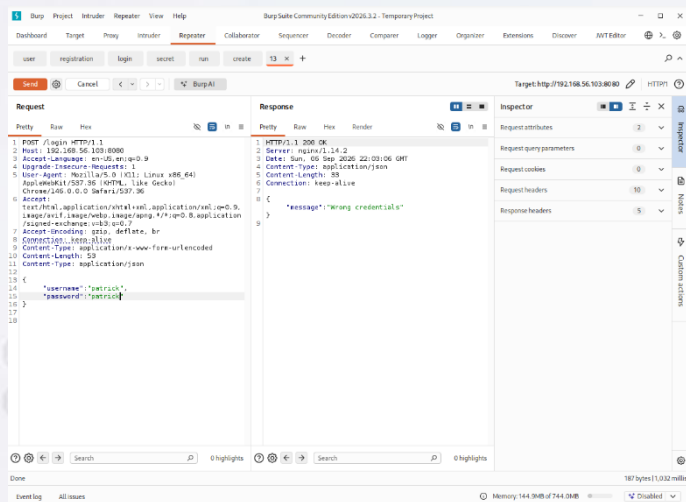
Burp Suite nos permite modificar las peticiones enviando el código de esta a la sección **Repeater** por lo cual enviaremos cada una de estas. La respuesta a esta petición nos está enviando el mensaje: El método no está permitido para la URL solicitada.



Cambiamos el método a **POST** dando clic derecho sobre el contenido de la petición, clic en la opción **Change request method**, clic a enviar, y ahora podemos ver en la respuesta el mensaje que necesitamos un nombre de usuario por lo cual probamos con el que conocemos “patrick”.

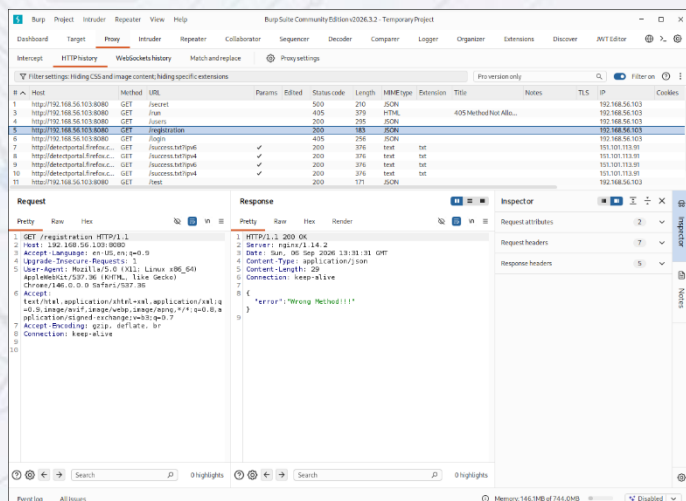


Damos el formato adecuado a nuestra petición agregando la línea 4 **Content-Type: application/json** de la respuesta al final de la petición abrimos y cerramos llaves y copiamos el formato del response indicando el nombre de usuario: **“username”:“patrick”**, la aplicación nos responde que el campo de contraseña no puede ir en blanco.

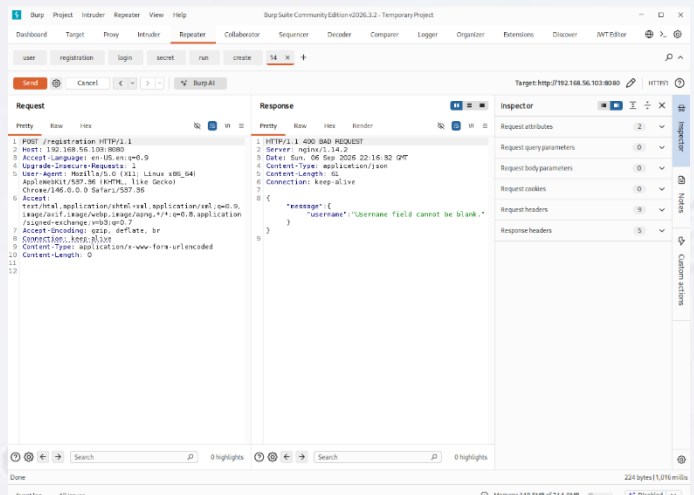


Ingresamos un valor de contraseña, nuevamente en el formato que está solicitando contraseña: **“password”:“patrick”**, podemos ver el comportamiento del api en busca de alguna vulnerabilidad, sin embargo, solo obtenemos el mensaje que las credenciales no son correctas.

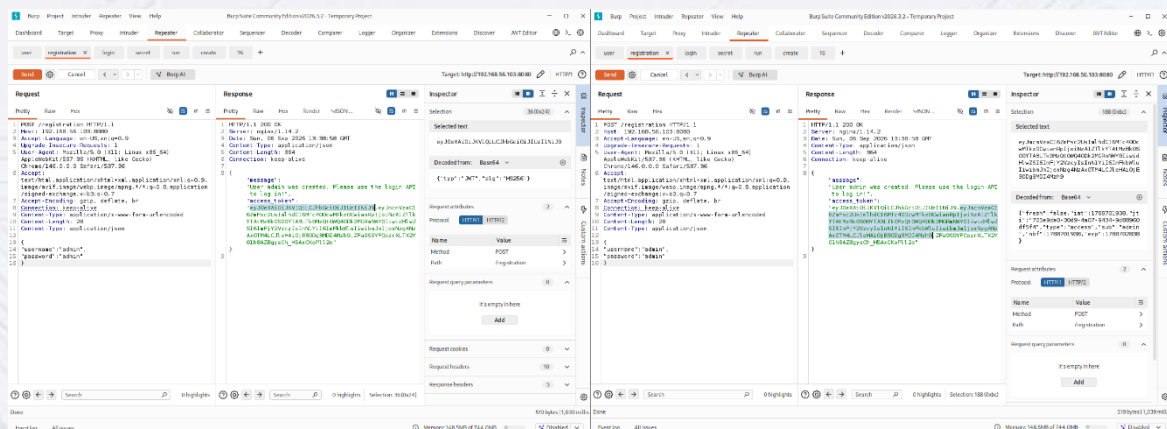
Ya que probamos que el usuario **“patrick”** que encontramos en la sección users hasta hora no sirve de mucho, probemos la sección **registration** la cual nos podría ayudar a crear un usuario.



La respuesta a esta petición nos está enviando el mensaje: Error en el método.



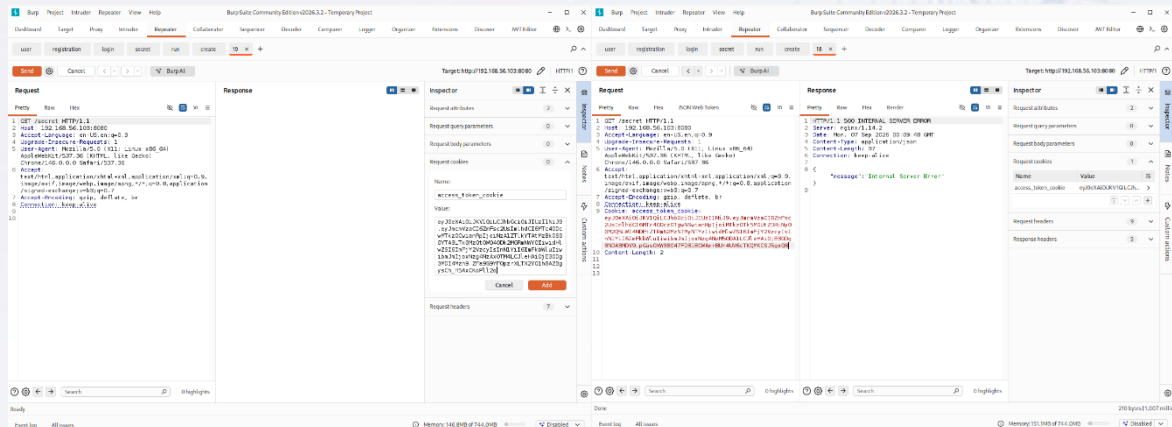
Al cambiar de método ahora nos pide al igual que el login un nombre de usuario y por lo tanto una contraseña y le daremos el formato solicitado. **“username”:**“admin”, **“password”:**“admin”.



Al enviar esta petición que el proceso está hecho sin pedir un tipo de usuario o permiso para crearlo y nos envía un **access_token** el cuales es **JWT (JSON Web Tokens)** este es un formato estandarizado donde se envía información del usuario como parte de autenticación y manejo de sesiones este se compone de 3 secciones el **header** que es la parte seleccionada de la imagen izquierda nos da información que es un **JWT** que se encripto en HS256. El **payload** que es la parte de la imagen derecha el cual nos da algunos datos del usuario sin embargo ninguno de estos datos incluye alguna información de relevancia y por último una **firma criptografica** la cual se podría descifrar para firmar nuestros propios **JWT** pero analizando las páginas del servidor web no interactúan con **JWT** externos solo da este como una identificación de sesión o **Cookie** que nos puede servir más adelante.

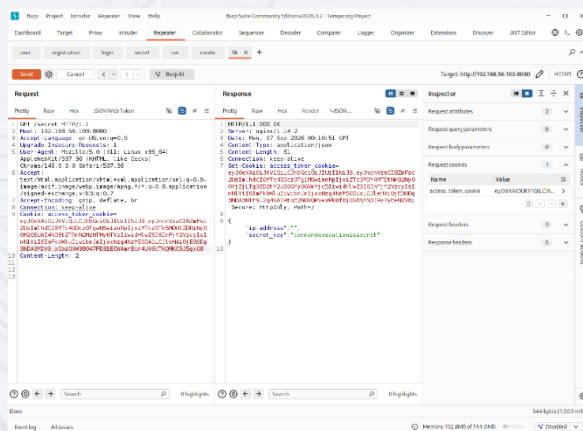
Partiendo de esta información podríamos verificar si el apartado **secret** requiere de un identificador de sesión

Burp Suite tiene la opción de agregar **Cookies** de la siguiente forma.



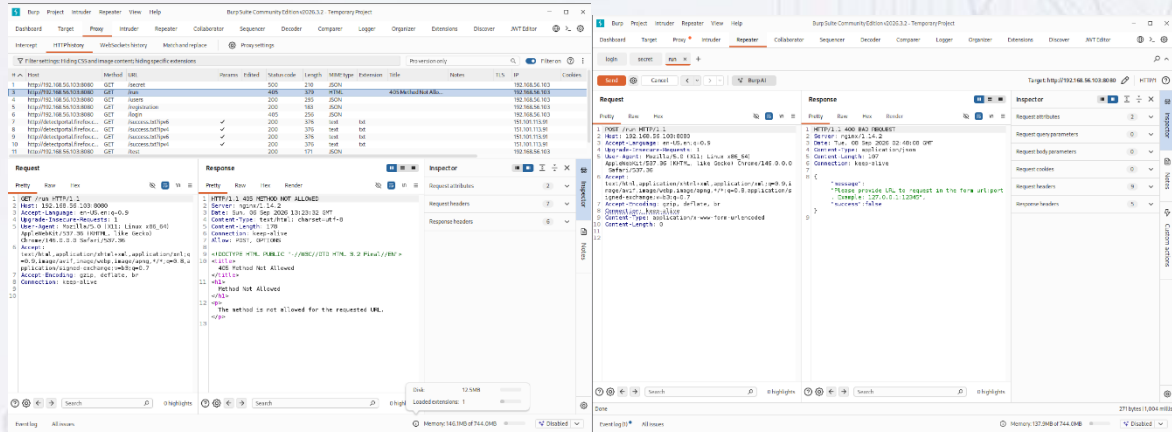
Buscamos del lado derecho el apartado **Request cookies** le damos en **Add** o simbolo **+** nos pedirá un nombre usaremos el valor **acces_token_cookie** y su valor introducimos la cadena de caracteres entre comillas y damos al botón **Add** estos datos son los que nos muestra al momento de hacer login con el usuario creado. Imagen izquierda.

Como se observa en la imagen de la derecha en la petición ya está nuestra cookie agregada solo nos resta dar clic en enviar.

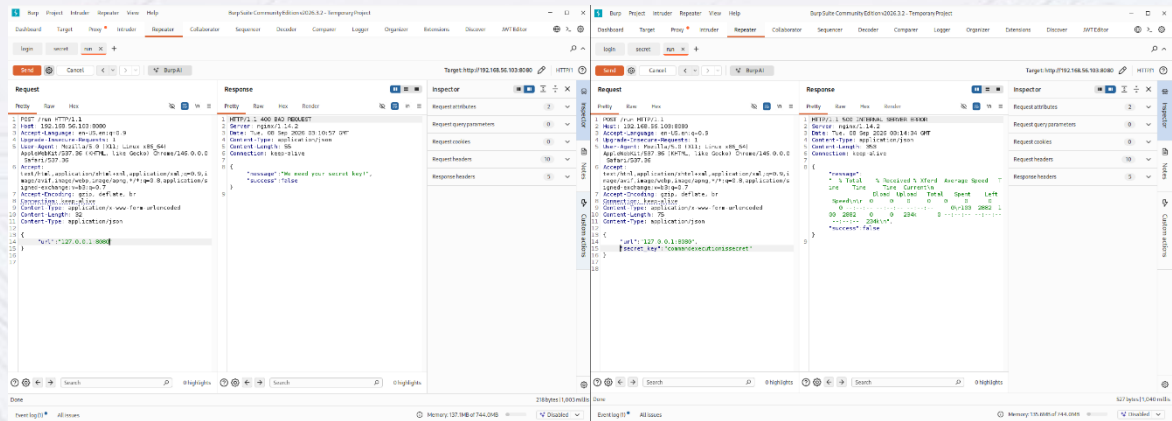


Nos podemos dar cuenta que el servidor no valida la información del usuario solo utiliza este token como llave para mostrar el contenido, por lo cual revela información que puede ser vital en este caso una **secret_key=commandexecutionissecret**.

Como podemos ver la contraseña por sí misma indica **execution** por lo cual podría tener algo que ver con el apartado **run**



Al analizar este nos indica que el método no está permitido por lo cual lo cambiamos, este responde que falta el parámetro **url** por lo cual le ingresaremos la dirección loopback **127.0.0.1** ya que queremos ver si se puede conectar consigo misma y el puerto **8080** que es por el cual estamos trabajando de la siguiente manera “url”:**“127.0.0.1:8080”**.



The screenshot shows the Burp Suite interface with a request and response view. The request is a GET to /api/v1/users/1234567890. The response is a 200 OK with a JSON body containing user details.

Request:

```

1 GET /api/v1/users/1234567890
2 Host: 192.168.56.103:8080
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0
6 (HTTP/1.1) 200 OK
7 Accept: application/json,application/javascript,application/xml;q=0.9,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
8 Content-Type: application/json
9 Content-Length: 71
10 Connection: keep-alive
11
12 {
13   "id": "1234567890",
14   "username": "johndoe",
15   "email": "john.doe@example.com",
16   "password": "securepassword123"
17 }
18

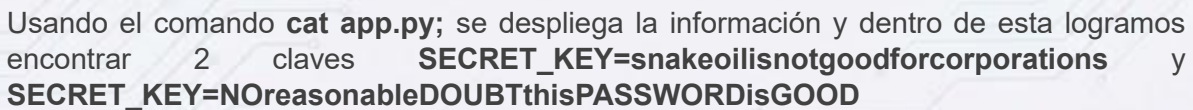
```

Response:

```

1 HTTP/1.1 200 OK
2 Server: Python/3.8.2
3 Date: Tue, 08 Sep 2020 05:40:00 GMT
4 Content-Type: application/json
5 Content-Length: 260
6 Connection: keep-alive
7
8 {
9   "message": "User details retrieved successfully."
10 }

```



```

patrick@SNAKEOIL:~$
Session Actions Edit View Help

kali@kali:~$
kali@kali:~$ ssh patrick@192.168.56.103
The authenticity of host '192.168.56.103 (192.168.56.103)' can't be established.
ED25519 key fingerprint is: SHA256:jeytSalC59gdH0E3qXAHakY3XaaSGl9IeOeEML7+Inc
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.103' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@192.168.56.103's password:
Permission denied, please try again.
patrick@192.168.56.103's password:
linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$ ls
Desktop Downloads local.txt Pictures Public Videos
Documents flask_blog Music Postman Templates vmware-tools-distrib
patrick@SNAKEOIL:~$ cat local.txt
cat: local.txt: No such file or directory
patrick@SNAKEOIL:~$ cat local.txt
local shell access obtained!
patrick@SNAKEOIL:~$

```

Usando la contraseña **“NoreasonableDOUBTthisPASSWORDisGOOD”** y el usuario **“patrick”** obtenemos acceso local en la maquina Snake oíl.

```
patrick@SNAKEOIL:~$ sudo su
[sudo] password for patrick:
root@SNAKEOIL:/home/patrick# ls
Desktop  Downloads  local.txt  Pictures  Public  Videos
Documents flask blog  Music     Postman   Templates vmware-tools-distrib
root@SNAKEOIL:/home/patrick# cd /
root@SNAKEOIL:/# ls
bin  dev  home  initrd.img.old  lib32  libx32  media  opt  root /sbin  sys  usr  vmlinuz
boot  etc  initrd.img  lib  lib64  lost-found  mnt  proc  run  srv  tmp  var  vmlinuz.old
root@SNAKEOIL:/# ls -la
total 72
drwxr-xr-x 19 root root 4096 Jun 12 2021 .
drwxr-xr-x 19 root root 4096 Jun 12 2021 ..
lrwxrwxrwx 1 root root 7 Jun 12 2021 bin -> usr/bin
drwxr-xr-x 3 root root 4096 Jun 12 2021 boot
drwx----- 2 root root 4096 Jun 12 2021 cache
drwxr-xr-x 17 root root 3880 Sep 8 13:01 dev
drwxr-xr-x 122 root root 4096 Sep 4 10:56 etc
drwxr-xr-x 3 root root 4096 Jun 12 2021 home
lrwxrwxrwx 1 root root 31 Jun 12 2021 initrd.img -> boot/initrd.img-4.19.0-16-amd64
lrwxrwxrwx 1 root root 31 Jun 12 2021 initrd.img.old -> boot/initrd.img-4.19.0-16-amd64
lrwxrwxrwx 1 root root 7 Jun 12 2021 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Jun 12 2021 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 Jun 12 2021 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 10 Jun 12 2021 libx32 -> usr/libx32
drwx----- 2 root root 16384 Jun 12 2021 lost-found
drwxr-xr-x 3 root root 4096 Jun 12 2021 media
drwxr-xr-x 3 root root 4096 Jun 12 2021 mnt
drwxr-xr-x 3 root root 4096 Jun 12 2021 opt
dr-xr-xr-x 178 root root 0 Sep 8 13:02 proc
drwx----- 4 root root 4096 Aug 16 2021 root
drwxr-xr-x 24 root root 780 Sep 8 14:15 run
lrwxrwxrwx 1 root root 8 Jun 12 2021/sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Jun 12 2021 srv
dr-xr-xr-x 13 root root 0 Sep 8 14:13 sys
drwxrwxrwt 14 root root 4096 Sep 8 13:02 tmp
drwxr-xr-x 14 root root 4096 Jun 12 2021 usr
drwxr-xr-x 12 root root 4096 Jun 19 2021 var
lrwxrwxrwx 1 root root 28 Jun 12 2021 vmlinuz -> boot/vmlinuz-4.19.0-16-amd64
lrwxrwxrwx 1 root root 28 Jun 12 2021 vmlinuz.old -> boot/vmlinuz-4.19.0-16-amd64
root@SNAKEOIL:/#
```

Ya que estamos dentro de la maquina solo resta tener acceso a “root” por lo cual ejecutamos el comando **sudo su** el usuario cambia a root por lo cual tenemos control de la máquina.

Vulnerabilidades encontradas después del pentesting

- La página no usa https por lo cual las peticiones y repuestas pudieron ser observadas y manipuladas.
- Usar nombres comunes para directorios web permite identificar que hace cada sección.
- Manejo inapropiado de errores estos nos dan información de lo que necesita para que el api nos responda, aunque el usuario no sea autorizado.
- Creación de usuarios no establece ningún control en lo que puede y no puede hacer un usuario.
- Acceso a contraseñas en el servidor web estas claves no deberían estar dentro de los archivos del sitio web.
- Acceso a contraseñas solo con un token de sesión sin validar su uso o el usuario que la creo.
- Permitir correr comandos del sistema y mostrar su ejecución dentro del sitio web.
- Reutilización de credenciales podemos encontrar credenciales de usuario y contraseñas redundantes que nos permitieron tomar acceso al sistema.

Conclusiones

Las pruebas de pentesting son primordiales para la puesta en servicio de un sitio web seguro, mitigando vulnerabilidades en las aplicaciones de desarrollo, la organización de nuestras credenciales y el tipo de acceso que le permitimos al usuario final y a cualquier usuario que de manera mal intencionada quiera hacerse de información relevante o apropiarse de nuestro sistema.

01100011 01111001 01100010
01100101 01110010...

01100011 01111001 01100010 01100101 0111001
00
01111001 01100010 01100111 0111001
011000011 01110001 011101
01111011 01100011 01100100 0111001
01100100 0111001

01111010 01110010 01100010